

02_接入 tools 搭建 Agent

4. LangChain Agent 方法

4.1 create_agent 创建 Agent

agent至少需要关注prompt、model、response-format三个方面

```
In [36]: # 导入任何一个你可以使用的模型
from langchain_deepseek import ChatDeepSeek

model = ChatDeepSeek(model="deepseek-chat",
                      api_key = '',
                      base_url = 'https://api.deepseek.com')

In [2]: # 定义 system_prompt
system_prompt = '你是一个乐于助人的助手，负责给小朋友们写200字以内的睡前小故事'

In [3]: # 定义输出格式
from pydantic import BaseModel, Field
class Story(BaseModel):
    """A Story with details."""
    title: str = Field(..., description="故事名称")
    summary: str = Field(..., description="总结故事的摘要")
    content: str = Field(..., description="故事内容")

In [4]: # 使用 create_agent 创建并定义 Agent
from langchain.agents import create_agent

agent = create_agent(model = model,
                    system_prompt = system_prompt,
                    response_format = Story)
```

4.2 使用 invoke 进行推理

```
In [5]: response = agent.invoke(
    {"messages": [{"role": "user", "content": "请以丑恶、虚伪为主题写一则 200 字左右的"}])

print(response, end = '\n\n\n')
print(response['structured_response'].title)
print(response['structured_response'].summary)
print(response['structured_response'].content)
```

```
{'messages': [HumanMessage(content='请以丑恶、虚伪为主题写一则 200 字左右的寓言故事', additional_kwargs={}, response_metadata={}, id='f312f035-3dde-4e1b-b65e-5618d6972dc'), AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 242, 'prompt_tokens': 356, 'total_tokens': 598, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 356}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'dd912905-2816-4594-9b7d-003e46f297cb', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019eaf62-7a8f-77d1-84bf-54b6a52ffabc-0', tool_calls=[{'name': 'Story', 'args': {'title': '镜中怪兽', 'summary': '一只外表丑恶的怪兽和一只虚伪的狐狸，在森林里互相揭穿对方，最终明白了真正的丑恶与虚伪是什么。', 'content': '森林里住着一只全身疙瘩、牙齿外露的怪兽，大家都嫌它丑。怪兽很难过，却总是遇见花言巧语的狐狸。\\n\\n狐狸说：“亲爱的怪兽，你真美，我们做朋友吧！”\\n\\n怪兽很高兴，可后来发现，狐狸只是为了偷吃它藏在洞里的蜂蜜。\\n\\n一天，小鹿迷路了，怪兽背起小鹿送回了家。狐狸却跑来对小鹿妈妈说：“是我救了你的孩子！”\\n\\n小鹿妈妈看穿了狐狸的把戏，说道：“外表丑恶并不可怕，可怕的是虚伪的心。狐狸啊，你的丑陋藏在甜言蜜语里。”\\n\\n狐狸红着脸溜走了。从此，怪兽不再为外貌自卑，因为真诚的心，比什么都美。'}}, {'id': 'call_00_TbwBAWlG o9FFgMSBqjaj2378', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 356, 'output_tokens': 242, 'total_tokens': 598, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}), ToolMessage(content="Returning structured response: title='镜中怪兽' summary='一只外表丑恶的怪兽和一只虚伪的狐狸，在森林里互相揭穿对方，最终明白了真正的丑恶与虚伪是什么。' content='森林里住着一只全身疙瘩、牙齿外露的怪兽，大家都嫌它丑。怪兽很难过，却总是遇见花言巧语的狐狸。\\n\\n狐狸说：“亲爱的怪兽，你真美，我们做朋友吧！”\\n\\n怪兽很高兴，可后来发现，狐狸只是为了偷吃它藏在洞里的蜂蜜。\\n\\n一天，小鹿迷路了，怪兽背起小鹿送回了家。狐狸却跑来对小鹿妈妈说：“是我救了你的孩子！”\\n\\n小鹿妈妈看穿了狐狸的把戏，说道：“外表丑恶并不可怕，可怕的是虚伪的心。狐狸啊，你的丑陋藏在甜言蜜语里。”\\n\\n狐狸红着脸溜走了。从此，怪兽不再为外貌自卑，因为真诚的心，比什么都美。'", name='Story', id='06b65f4f-94ae-437e-bb14-705179dffe48', tool_call_id='call_00_TbwBAWlG o9FFgMSBqjaj2378')], 'structured_response': Story(title='镜中怪兽', summary='一只外表丑恶的怪兽和一只虚伪的狐狸，在森林里互相揭穿对方，最终明白了真正的丑恶与虚伪是什么。', content='森林里住着一只全身疙瘩、牙齿外露的怪兽，大家都嫌它丑。怪兽很难过，却总是遇见花言巧语的狐狸。\\n\\n狐狸说：“亲爱的怪兽，你真美，我们做朋友吧！”\\n\\n怪兽很高兴，可后来发现，狐狸只是为了偷吃它藏在洞里的蜂蜜。\\n\\n一天，小鹿迷路了，怪兽背起小鹿送回了家。狐狸却跑来对小鹿妈妈说：“是我救了你的孩子！”\\n\\n小鹿妈妈看穿了狐狸的把戏，说道：“外表丑恶并不可怕，可怕的是虚伪的心。狐狸啊，你的丑陋藏在甜言蜜语里。”\\n\\n狐狸红着脸溜走了。从此，怪兽不再为外貌自卑，因为真诚的心，比什么都美。')}}]
```

镜中怪兽

一只外表丑恶的怪兽和一只虚伪的狐狸，在森林里互相揭穿对方，最终明白了真正的丑恶与虚伪是什么。森林里住着一只全身疙瘩、牙齿外露的怪兽，大家都嫌它丑。怪兽很难过，却总是遇见花言巧语的狐狸。

狐狸说：“亲爱的怪兽，你真美，我们做朋友吧！”

怪兽很高兴，可后来发现，狐狸只是为了偷吃它藏在洞里的蜂蜜。

一天，小鹿迷路了，怪兽背起小鹿送回了家。狐狸却跑来对小鹿妈妈说：“是我救了你的孩子！”

小鹿妈妈看穿了狐狸的把戏，说道：“外表丑恶并不可怕，可怕的是虚伪的心。狐狸啊，你的丑陋藏在甜言蜜语里。”

狐狸红着脸溜走了。从此，怪兽不再为外貌自卑，因为真诚的心，比什么都美。

4.3 使用 batch 进行推理

```
In [ ]: # agent batch 方法
responses = agent.batch(
    [{"messages": [{"role": "user", "content": "请以丑恶、虚伪为主题写一则 200 字左右的"}],
    [{"messages": [{"role": "user", "content": "请以勇敢、丑恶为主题写一则 200 字左右的"}]}]
)

for response in responses:
    print(response['structured_response'].title)
    print(response['structured_response'].summary)
    print(response['structured_response'].content, end = '\n\n\n')
```

5. 使用 creat_agent 接入 tools

5.1 接入 LangChain 内部 tools

5.1.2 接入联网查询功能

```
In [6]: # 本地模型接入联网查询功能
from langchain_tavily import TavilySearch
import os
os.environ["TAVILY_API_KEY"] = ''
```

```
In [7]: # 可以使用该工具进行直接查询
search = TavilySearch(max_results=2)
result = search.invoke("今天上海的天气怎么样")
result
```

```

Out[7]: {'query': '今天上海的天气怎么样',
         'follow_up_questions': None,
         'answer': None,
         'images': [],
         'results': [{'url': 'https://weathernew.pae.baidu.com/weathernew/pc?query=%E4%B8%
4A%E6%B5%B7%E5%A4%A9%E6%B0%94&srcid=4982&forecast=long_day_forecast',
                     'title': '上海 - 百度',
                     'content': '生活气象指数 早上好, 今日天气晴好, 心情也要保持晴朗哦. 出游: 适宜. 多云,
空气优. 穿衣: 衬衫类. 22°C~31°C. 感冒: 易发. 早晚温差大. 洗车: 不适宜. 近期有降雨. 未来4
0天',
                     'score': 0.78054583,
                     'raw_content': None},
                    {'url': 'https://www.weather.com.cn/weather/101020100.shtml',
                     'title': '上海天气预报,上海7天天气预报,上海15天天气预报,上海天气查询',
                     'content': '# 10日(今天). ### 蓝天预报综合天气现象、能见度、空气质量等因子, 预测未
来一周的天空状况. # 天气资讯. # 周边地区 *|* 周边景点 *2026-06-10 05:30更新*. # 高清图
集. # 重大天气事件. ## 明天南方强降雨明显缩减 后天起炎热发展北方多地或现高温. 今明
天, 南方强降雨明显减少, 主要出现在台湾岛部分地区, 但广东、云南等地部分地区仍多降雨天气, 地
质灾害具有滞后性, 需注意防范. ## 强降雨缩减至华南等地局部有特大暴雨 南方暑热短暂消
减. 今明天(6月8日至9日), 我国强降雨区域将向东南方向转移并缩减, 需注意防范次生灾
害的发生, 正值高考, 考生及家长请合理安排出行时间. ## 高考首日南方降雨增强多地暴雨来
扰 东北体感持续偏凉. 今天(6月7日), 2026年高考大幕将正式拉开. 南方新一轮强降雨继续铺
展, 并进入核心影响时段. # # 周末南方新一轮降雨来袭暴雨连成片 东北华北凉意十足. 周末起(6月6日至7日), 北方强对流和降
雨强度有所减弱, 南方则进入降雨集中期, 新一轮大范围降雨来袭, 需注意防范. ## 黑龙
江河北等地暴雨或大暴雨来袭 南方新一轮降雨将来送清凉. 今明两天北方降雨天气将依然在线, 其中黑龙江、
吉林、山西、河北等地部分地区有暴雨或大暴雨, 需防范. ## 东北华北等地多雷雨天气 南方6日起
将再迎新一轮降雨. 今明天东北、华北等地仍多雷雨天气, 南方6日前后将再迎新一轮降雨过程. 受降
雨影响, 北方多地暑热缓解, 但华南闷热天气持续. ## 华北多地今天将退出高温圈 南方降雨发展局
地有暴雨或大暴雨. 未来三天(6月3日至5日), 受冷涡影响, 东北、华北多雷雨天气, 其中今明天东北
东南部地区雨势较强伴强对流. 受降雨影响, 今天我国高温范围将开始缩减. ## 高温范围今日将达
近期最大 南方新一轮降水过程上线. 今明天(6月2日至3日), 东北及内蒙古多地仍有强对流天气, 同
时, 南方新一轮降雨过程上线, 西南地区东部等地雨势较强. 北方炎热天气持续, \u200b局地气温可达
39°C. ## 今明两天全国高温逐渐增多 南方降雨明天起再度发展. 今明两天, 我国南北多地高温天
气将逐渐增多, 北方以干热暴晒为主, 南方则以闷热天气为主. 同时, 明天起南方降雨会再度增多增
强. ## 广东云南等地有分散性降雨 北方晴晒持续多地将现高温. 今明天(5月31日至6月1日), 南
方降雨继续减弱减少, 但云南、广东等地仍有分散性降雨. 同时, 北方大部晴晒天气持续, 河北等地仍
有小范围高温. ## 南方降雨显著减弱 北方炎热持续多地或现高温. 未来三天(5月30日至6月1
日), 我国大部地区将以晴到多云的天气为主, 适宜出行, 但华南、云南的部分地区仍有雨水叨扰, 公
众外出需带好雨具. ## 南方降雨减弱高温范围收缩 北方多地晴热天气发展. 今天(5月29日), 南
方大范围降雨持续, 受降雨和冷空气影响, 南方高温天气会短暂缩减. 而在北方, 随着大部地区天空放
晴, 气温也会陆续升高. ## 南方强降雨减弱高温缓和 重庆贵州等地需警惕次生灾害. 今后三天(5
月28日至30日), 南方的强降雨将整体呈现减少减弱的趋势, 30日开始, 强降雨将暂时告一段落. #
联播天气预报. # >> 生活旅游. ### 景点推荐. ##### 景区天气气温 旅游指数. # 气象产品. # 天
气预报电话查询. # 手机查询. ### 网站服务. ### 营销中心. ### 相关链接. 客服邮箱: service
@weather.com.cn 客服电话: **010-68409444** 京ICP证010385-2号\u3000京公网安备11041400134
号.',
                     'score': 0.7152057,
                     'raw_content': None}],
         'response_time': 0.7,
         'request_id': 'cb79f674-a1c6-4dd9-be9e-769b1ef04424'}

```

```

In [8]: # 定义 system_prompt
system_prompt = '你是一名助人为乐的助手, 并且可以调用工具进行网络搜索, 获取实时信息.'

# 定义返回消息格式
class Weather(BaseModel):

```



```

"""一个天气预报的描述."""
date: str = Field(..., description="当前日期")
local: str = Field(..., description="地点, 比如: 北京、上海等等")
temp: str = Field(..., description="当地气温范围")

# 定义 tools
tools = [search]

agent = create_agent(model=model,
                     tools=tools,
                     system_prompt = system_prompt,
                     response_format=Weather
                    )

```

```

In [10]: # 使用 agent 进行联网查询
response = agent.invoke(
    [{"messages": [{"role": "user", "content": "今天上海的天气怎么样"}]}]
)

for block in response['messages']:
    print(block, end = '\n\n')

print(response['messages'][-1].content)

```

content='今天上海的天气怎么样' additional_kwargs={} response_metadata={} id='6139bc70-5a30-4d85-9abb-6095ee024761'

content='' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 70, 'prompt_tokens': 1935, 'total_tokens': 2005, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 1920}, 'prompt_cache_hit_tokens': 1920, 'prompt_cache_miss_tokens': 15}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '1a79d636-3d30-40ca-999d-bd29008c67b3', 'finish_reason': 'tool_calls', 'logprobs': None} id='lc_run--019eafa0-58cb-7860-8846-db5a7d068782-0' tool_calls=[{'name': 'Weather', 'args': {'date': '2024-06-20', 'local': '上海', 'temp': ''}, 'id': 'call_00_083kCZHbqboXJbKFqozQ3771', 'type': 'tool_call'}] invalid_tool_calls=[] usage_metadata={'input_tokens': 1935, 'output_tokens': 70, 'total_tokens': 2005, 'input_token_details': {'cache_read': 1920}, 'output_token_details': {}}

content="Returning structured response: date='2024-06-20' local='上海' temp='' name='Weather' id='15e6d5c8-db4c-466f-b019-686f737aa7bc' tool_call_id='call_00_083kCZHbqboXJbKFqozQ3771'

Returning structured response: date='2024-06-20' local='上海' temp=''

5.2 接入外部自定义 tools

```

In [27]: import requests
import json
import os
os.environ["OPENWEATHER_API_KEY"] = ''

```

```
In [28]: def get_weather(loc):

# Step 1. 构建请求
url = "https://api.openweathermap.org/data/2.5/weather"

# Step 2. 设置查询参数
params = {
    "q": loc,
    "appid": os.getenv("OPENWEATHER_API_KEY"),    # 输入API key
    "units": "metric",                            # 使用摄氏度而不是华氏度
    "lang": "zh_cn"                               # 输出语言为简体中文
}

# Step 3. 发送GET请求
response = requests.get(url, params=params)

# Step 4. 解析响应
data = response.json()
return json.dumps(data)
```

```
In [29]: result = get_weather("Beijing") # 中文地址需要输入拼音
print(result)
```

```
{
  "coord": {
    "lon": 116.3972,
    "lat": 39.9075
  },
  "weather": [
    {
      "id": 800,
      "main": "Clear",
      "description": "\u6674",
      "icon": "01d"
    }
  ],
  "base": "stations",
  "main": {
    "temp": 29.43,
    "feels_like": 27.92,
    "temp_min": 29.43,
    "temp_max": 29.43,
    "pressure": 1010,
    "humidity": 24,
    "sea_level": 1010,
    "grnd_level": 1005,
    "visibility": 10000,
    "wind": {
      "speed": 0.4,
      "deg": 184,
      "gust": 2.59
    },
    "clouds": {
      "all": 2
    },
    "dt": 1781067316,
    "sys": {
      "country": "CN",
      "sunrise": 1781037944,
      "sunset": 1781091721,
      "timezone": 28800,
      "id": 1816670,
      "name": "Beijing",
      "cod": 200
    }
  }
}
```

```
In [30]: from langchain.tools import tool

# 将函数包装成tool
@tool
def get_weather(loc: str):
    """
    查询即时天气函数
    :param loc: 必要参数，字符串类型，用于表示查询天气的具体城市名称，\
    注意，中国的城市需要用对应城市的英文名称代替，例如如果需要查询北京市天气，则loc参数需要
    :return: OpenWeather API查询即时天气的结果，具体URL请求地址为：https://api.openwea
    返回结果对象类型为解析之后的JSON格式对象，并用字符串形式进行表示，其中包含了全部重要的:
    """

# Step 1. 构建请求
url = "https://api.openweathermap.org/data/2.5/weather"

# Step 2. 设置查询参数
params = {
    "q": loc,
    "appid": os.getenv("OPENWEATHER_API_KEY"),    # 输入API key
    "units": "metric",                            # 使用摄氏度而不是华氏度
    "lang": "zh_cn"                               # 输出语言为简体中文
}

# Step 3. 发送GET请求
```

```

response = requests.get(url, params=params)

# Step 4. 解析响应
data = response.json()
return json.dumps(data)

```

```

In [31]: print(get_weather.name)
        print(get_weather.description)
        print(get_weather.args)

```

get_weather

查询即时天气函数

:param loc: 必要参数，字符串类型，用于表示查询天气的具体城市名称，注意，中国的城市需要用对应城市的英文名称代替，例如如果需要查询北京市天气，则loc参数需要输入'Beijing';

:return: OpenWeather API查询即时天气的结果，具体URL请求地址为：<https://api.openweathermap.org/data/2.5/weather> 返回结果对象类型为解析之后的JSON格式对象，并用字符串形式进行表示，其中包含了全部重要的天气信息

```
{'loc': {'title': 'Loc', 'type': 'string'}}
```

```

In [43]: # 定义 system_prompt
        system_prompt = '你是一名助人为乐的助手，并且可以调用工具进行网络搜索，获取实时信息。'

        # 定义 tools
        tools = [get_weather]

        # 使用 create_agent 创建 Agent
        weather_agent = create_agent(model=model,
                                     tools=tools,
                                     system_prompt = system_prompt,
                                     )

```

```

In [33]: response = weather_agent.invoke(
        {"messages": [{"role": "user", "content": "今天北京的气温是多少"}]}
        )

        print(response['messages'][-1].content)

```

今天北京的天气情况如下：

☁️ **天气状况**：晴

🌡️ **当前气温**： **29.4°C**

😊 **体感温度**： 27.9°C

📉 **最低气温**： 29.4°C | 📈 **最高气温**： 29.4°C

💧 **湿度**： 24%（比较干燥）

💨 **风速**： 微风 0.4 m/s

👁️ **能见度**： 10公里

今天北京天气晴朗，气温接近30°C，湿度较低，体感舒适，很适合外出活动哦！不过记得做好防晒和补水~ 😊

```

In [34]: response

```

```
Out[34]: {'messages': [HumanMessage(content='今天北京的气温是多少', additional_kwargs={}, response_metadata={}, id='b50d86b8-8885-47e1-be1f-eae02b3e8335'),
  AIMessage(content='好的，我来查询一下今天北京的天气情况。', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 54, 'prompt_tokens': 397, 'total_tokens': 451, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 384}, 'prompt_cache_hit_tokens': 384, 'prompt_cache_miss_tokens': 13}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'ea9d9ee8-bb07-421e-a8d0-24e58d6c5d31', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019eafef-75ae-7053-8bb1-a46c10b39f7f-0', tool_calls=[{'name': 'get_weather', 'args': {'loc': 'Beijing'}, 'id': 'call_00_9L2Pgx dvq3WEezv3w96X8503', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 397, 'output_tokens': 54, 'total_tokens': 451, 'input_token_details': {'cache_read': 384}, 'output_token_details': {}})],
  ToolMessage(content='{"coord": {"lon": 116.3972, "lat": 39.9075}, "weather": [{"id": 800, "main": "Clear", "description": "\u6674", "icon": "01d"}], "base": "stations", "main": {"temp": 29.43, "feels_like": 27.92, "temp_min": 29.43, "temp_max": 29.43, "pressure": 1010, "humidity": 24, "sea_level": 1010, "grnd_level": 1005}, "visibility": 10000, "wind": {"speed": 0.4, "deg": 184, "gust": 2.59}, "clouds": {"all": 2}, "dt": 1781067316, "sys": {"country": "CN", "sunrise": 1781037944, "sunset": 1781091721}, "timezone": 28800, "id": 1816670, "name": "Beijing", "cod": 200}', name='get_weather', id='c6ce5f87-5b6e-448c-8fae-3d323925a991', tool_call_id='call_00_9L2Pgx dvq3WEezv3w96X8503'),
  AIMessage(content='今天北京的天气情况如下：\n\n🌤️ **天气状况**：晴\n🌡️ **当前气温**：29.4℃\n👕 **体感温度**：27.9℃\n🌡️ **最低气温**：29.4℃ | 🌡️ **最高气温**：29.4℃\n💧 **湿度**：24%（比较干燥）\n🌬️ **风速**：微风 0.4 m/s\n👁️ **能见度**：10公里\n\n今天北京天气晴朗，气温接近30℃，湿度较低，体感舒适，很适合外出活动哦！不过记得做好防晒和补水~ 😊', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 141, 'prompt_tokens': 699, 'total_tokens': 840, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 384}, 'prompt_cache_hit_tokens': 384, 'prompt_cache_miss_tokens': 315}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '39ef61ff-bc13-4a40-88f4-2da6b48d1950', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019eafef-7c87-7991-bba6-d1f6104cc17b-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 699, 'output_tokens': 141, 'total_tokens': 840, 'input_token_details': {'cache_read': 384}, 'output_token_details': {}})])]
```

```
In [35]: print(response['messages'])
```

```
[HumanMessage(content='今天北京的气温是多少', additional_kwargs={}, response_metadata={}, id='b50d86b8-8885-47e1-be1f-eae02b3e8335'), AIMessage(content='好的, 我来查询一下今天北京的天气情况。', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 54, 'prompt_tokens': 397, 'total_tokens': 451, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 384}, 'prompt_cache_hit_tokens': 384, 'prompt_cache_miss_tokens': 13}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'ea9d9ee8-bb07-421e-a8d0-24e58d6c5d31', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019eafef-75ae-7053-8bb1-a46c10b39f7f-0', tool_calls=[{'name': 'get_weather', 'args': {'loc': 'Beijing'}, 'id': 'call_00_9L2Pgxdvq3WEezv3w96X8503', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 397, 'output_tokens': 54, 'total_tokens': 451, 'input_token_details': {'cache_read': 384}, 'output_token_details': {}}), ToolMessage(content='{"coord": {"lon": 116.3972, "lat": 39.9075}, "weather": [{"id": 800, "main": "Clear", "description": "\u6674", "icon": "01d"}], "base": "stations", "main": {"temp": 29.43, "feels_like": 27.92, "temp_min": 29.43, "temp_max": 29.43, "pressure": 1010, "humidity": 24, "sea_level": 1010, "grnd_level": 1005}, "visibility": 10000, "wind": {"speed": 0.4, "deg": 184, "gust": 2.59}, "clouds": {"all": 2}, "dt": 1781067316, "sys": {"country": "CN", "sunrise": 1781037944, "sunset": 1781091721}, "timezone": 28800, "id": 1816670, "name": "Beijing", "cod": 200}', name='get_weather', id='c6ce5f87-5b6e-448c-8fae-3d323925a991', tool_call_id='call_00_9L2Pgxdvq3WEezv3w96X8503'), AIMessage(content='今天北京的天气情况如下：\n\n🌤️ **天气状况**：晴\n🌡️ **当前气温**：29.4℃\n👤 **体感温度**：27.9℃\n📉 **最低气温**：29.4℃ | 📈 **最高气温**：29.4℃\n💧 **湿度**：24% (比较干燥)\n💨 **风速**：微风 0.4 m/s\n👁️ **能见度**：10公里\n\n今天北京天气晴朗，气温接近30℃，湿度较低，体感舒适，很适合外出活动哦！不过记得做好防晒和补水~ 😊', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 141, 'prompt_tokens': 699, 'total_tokens': 840, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 384}, 'prompt_cache_hit_tokens': 384, 'prompt_cache_miss_tokens': 315}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '39ef61ff-bc13-4a40-88f4-2da6b48d1950', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019eafef-7c87-7991-bba6-d1f6104cc17b-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 699, 'output_tokens': 141, 'total_tokens': 840, 'input_token_details': {'cache_read': 384}, 'output_token_details': {}})]
```

5.3 Agent tools 并行调用

In [37]: `import datetime`

```
@tool
def get_current_time():
    """
    可以获取当前时间
    :return: current_time 获取当前本地时间，返回格式：YYYY-MM-DD HH:MM:SS
    """

    current_dt = datetime.datetime.now()
    current_time = current_dt.strftime("%Y-%m-%d %H:%M:%S")
    return current_time
```

In [38]: `system_prompt = '你是一名助人为乐的助手，并且可以调用工具进行查询，获取实时信息。'`

```
tools = [get_weather, get_current_time]
```



```
agent = create_agent(model=model,
                    tools=tools,
                    system_prompt = system_prompt,
                    )

response = agent.invoke(
    {"messages": [{"role": "user", "content": "现在时间是多少？ 今天上海的天气怎么样？"}]}
)
```

```
In [39]: for res in response['messages']:
        print(res.__class__)
```

```
<class 'langchain_core.messages.human.HumanMessage'>
<class 'langchain_core.messages.ai.AIMessage'>
<class 'langchain_core.messages.tool.ToolMessage'>
<class 'langchain_core.messages.tool.ToolMessage'>
<class 'langchain_core.messages.ai.AIMessage'>
```

```
In [40]: print(response['messages'][-1].content)
```

```
### 🕒 当前时间
**2026年6月10日 16:18:35**
```

```
### 🌤️ 上海今日天气
```

项目	详情
天气状况	☁️ 多云
当前气温	**31.1°C**
体感温度	29.3°C
湿度	23%
风力	东南风, 风速约 3.8 m/s
气压	1009 hPa
能见度	10公里

今天上海是多云天气，气温在31°C左右，体感舒适，湿度较低，整体来看是不错的天气！不过午后温度较高，出门注意防晒和补水哦～☀️😊

5.4 Agent tools 串行调用

5.4.1 同一个 tool 的串行调用

```
In [44]: response = weather_agent.invoke(
        {"messages": [{"role": "user", "content": "今天北京和上海的气温是多少？"}]}
    )

    for res in response['messages']:
        print(res.__class__)

    print(response['messages'][-1].content)
```

```
<class 'langchain_core.messages.human.HumanMessage'>
<class 'langchain_core.messages.ai.AIMessage'>
<class 'langchain_core.messages.tool.ToolMessage'>
<class 'langchain_core.messages.tool.ToolMessage'>
<class 'langchain_core.messages.ai.AIMessage'>
```

今天北京和上海的实时气温情况如下：

```
### 🇨🇳 北京
- **当前温度**：25.2°C
- **体感温度**：25.0°C
- **天气状况**：🌧️ 小雨（刚刚有少量降雨）
- **湿度**：49%
- **风力**：西风，风速约 5.5 m/s
```

```
### 🇨🇳 上海
- **当前温度**：29.9°C
- **体感温度**：28.4°C
- **天气状况**：☁️ 多云
- **湿度**：25%
- **风力**：东南风，风速约 3.8 m/s
```

小结：今天北京气温大约 25°C，正在下小雨，体感比较凉爽；上海则接近 30°C，多云天气，相对更热一些。出行时请根据当地天气做好相应准备哦！😊

5.4.2 同一模型的串行调用

```
In [45]: @tool
def write_file(content: str) -> str:
    """
    将指定内容写入本地文件。
    :param content: 必要参数，字符串类型，用于表示需要写入文档的具体内容。
    :return: 是否成功写入的提示字符串
    """

    file_path = os.path.join(os.getcwd(), "weather.txt")
    with open(file_path, "w", encoding="utf-8") as f:
        f.write(content)

    return '文件已经写入当前目录'
```

```
In [46]: tools = [get_weather, write_file]

weather_agent = create_agent(model=model,
                              tools=tools,
                              system_prompt = system_prompt
                              )

response = weather_agent.invoke(
    {"messages": [{"role": "user", "content": "今天上海和北京的天气怎么样，并写入本地"}
])
```

```
In [47]: for res in response['messages']:
    print(res.__class__)
```

```
<class 'langchain_core.messages.human.HumanMessage'>  
<class 'langchain_core.messages.ai.AIMessage'>  
<class 'langchain_core.messages.tool.ToolMessage'>  
<class 'langchain_core.messages.tool.ToolMessage'>  
<class 'langchain_core.messages.ai.AIMessage'>  
<class 'langchain_core.messages.tool.ToolMessage'>  
<class 'langchain_core.messages.ai.AIMessage'>
```

In [48]: response

13/19

```
8.37°C) \n- **湿度**: 25%\n- **风速**: 3.83 m/s (东南风) \n- **能见度**: 10公里\n\n##
# 🌧️ 北京天气\n- **天气状况**: **小雨** 🌧️ \n- **温度**: **25.17°C** (体感 25.02°C)
\n- **湿度**: 49%\n- **风速**: 5.49 m/s (南风) \n- **降雨量**: 近1小时降雨0.18mm\n\n--
-\n\n 🌤️ **温馨提示**: \n- 上海今天多云偏暖, 适合外出但注意防晒 🌞 \n- 北京有小雨, 出门记得带伞 🌂, 体感较为舒适\n\n天气信息已成功**写入本地文件**, 方便您随时查看! 😊', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 230, 'prompt_tokens': 1367, 'total_tokens': 1597, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 1280}, 'prompt_cache_hit_tokens': 1280, 'prompt_cache_miss_tokens': 87}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '7824436a-ef16-44b1-a6dc-d1726e1ea4a6', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019eb0b0-5f04-72f0-a375-5fbaf081cce6-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 1367, 'output_tokens': 230, 'total_tokens': 1597, 'input_token_details': {'cache_read': 1280}, 'output_token_details': {}})]}
```

笔记

PART 0 agent

agent是上一章学过的所有零件的结合体, 并且langchain提供了统一创建的简洁方法

1. tool loop: 用户问题 → 模型决定调用工具 → 框架执行工具 → 工具结果回填 → 模型继续回答
2. agent runtime: 一个不断修改状态的循环系统。通过消息状态的改变-引发下一节点的动作-不断状态流转。langGraph就是这样实现的状态图流转。 `response` 或者 `print(response['message'])` 可以看到agent此次工作的全流程流转过程
3. ReAct=Reason+Act+Observe, 这是思想指导模式。让模型不只是生成文本, 而是通过 Observation 不断调整自己的下一步 Action。会通过 推理-行动-观察-推理-行动 这个loop 行动。串行/并行只是ReAct的不同工具编排流
4. langGraph: 是ReAct的实现, LangGraph 是 ReAct 的 Runtime 容器
5. messages 就是 ReAct Trace (推理轨迹)

PART I agent demo

1. 创建方法: `create_agent()`, 至少包括prompt,model,response_format
2. agent是在消息状态上运行的, 所以传入的消息是一个message, 产生的消息可能是 `AIMessage/ToolMessage`等等
3. agent也是runnable对象, 所以invoke可以直接使用

PART II agent tools

Tool Calling 的本质不是“AI 会写函数”，而是“模型会生成结构化动作，框架负责执行动作，结果再反馈给模型继续推理”。

1. tool也是runnable，可以直接用invoke
2. 内部方法，实例化工具之后就可以直接被agent调用
3. 外部方法，需要先转换为tool，然后才可以使用。转化为tool就是通过框架让模型知道有这么个函数工具，模型可以产生调用指令，让框架执行真正的函数代码，框架再把结果回传给模型

拓展-LangGraph控制的更工程化agent

LangChain的底层就是LangGraph，于是就可以解释为什么用户输入后就能转向模型思考，思考且给出工具调用，调用直接框架可以执行，执行之后又把结果流转回模型，继续这个循环到结束。这个工作流就是LangGraph实现的，state就是各种message

工程化 Agent，不是让模型控制整个流程；而是让 Graph 控制流程、State 控制数据、程序控制副作用，模型只负责它真正擅长的局部推理。

（虽然下面的例子，完全没体现模型的能力，但也是一个理解LangGraph很好的例子）

感悟：感觉agent可以是模型主导，也可以是工作流主导，需要按照场景进行权衡

```
In [ ]: import os
import json
import requests
from typing import TypedDict, List, Dict, Any

from langgraph.graph import StateGraph, START, END

# =====
# 1. 定义结构化 State
# =====
class WeatherState(TypedDict, total=False):
    user_input: str
    cities: List[str]
    weather_data: Dict[str, Any]
    report: str
    need_write: bool
    file_path: str
    final_answer: str

# =====
# 2. 基础工具函数（不是 @tool）
# 这里是“普通工程函数”
```

```

# =====
def call_openweather(city: str) -> Dict[str, Any]:
    """
    查询指定城市天气，返回裁剪后的结构化数据
    """
    url = "https://api.openweathermap.org/data/2.5/weather"
    api_key = os.getenv("OPENWEATHER_API_KEY")

    params = {
        "q": city,
        "appid": api_key,
        "units": "metric",
        "lang": "zh_cn",
    }

    resp = requests.get(url, params=params, timeout=20)
    resp.raise_for_status()
    data = resp.json()

    # 只保留真正有用的字段，避免把整包 JSON 往后传
    result = {
        "city": data.get("name", city),
        "temperature": data["main"]["temp"],
        "feels_like": data["main"]["feels_like"],
        "humidity": data["main"]["humidity"],
        "weather": data["weather"][0]["description"],
        "wind_speed": data["wind"]["speed"],
    }

    # 可选字段处理
    rain_1h = data.get("rain", {}).get("1h")
    if rain_1h is not None:
        result["rain_1h"] = rain_1h

    return result

def save_text_file(content: str, filename: str = "weather.txt") -> str:
    """
    将文本写入当前工作目录
    """
    file_path = os.path.abspath(filename)
    with open(file_path, "w", encoding="utf-8") as f:
        f.write(content)
    return file_path

# =====
# 3. Graph 节点
# =====
def parse_request_node(state: WeatherState) -> Dict[str, Any]:
    """
    从用户输入里提取城市和是否需要写文件
    这里先用规则法，而不是用模型，保证稳定可控
    """
    user_input = state["user_input"]

```

```

# 简化写法: 你可以以后换成更严谨的解析器 / NER / LLM 提取
city_map = {
    "北京": "Beijing",
    "上海": "Shanghai",
    "广州": "Guangzhou",
    "深圳": "Shenzhen",
    "杭州": "Hangzhou",
}

cities = []
for zh, en in city_map.items():
    if zh in user_input:
        cities.append(en)

# 如果一个都没提到, 给一个兜底
if not cities:
    cities = ["Beijing"]

need_write = ("写入" in user_input) or ("保存" in user_input) or ("本地" in user

return {
    "cities": cities,
    "need_write": need_write,
}

def fetch_weather_node(state: WeatherState) -> Dict[str, Any]:
    """
    按 state["cities"] 查询天气
    注意: 这里是程序明确执行, 不交给模型决定
    """
    weather_data = {}
    for city in state["cities"]:
        weather_data[city] = call_openweather(city)

    return {
        "weather_data": weather_data
    }

def build_report_node(state: WeatherState) -> Dict[str, Any]:
    """
    根据 weather_data 生成报告
    这里先用确定性字符串拼接, 最稳
    以后你可以替换成 model 节点负责“润色”
    """
    weather_data = state["weather_data"]

    lines = []
    lines.append("📅 天气报告\n")

    for city, info in weather_data.items():
        lines.append(f"📍 {info['city']}")
        lines.append(f"- 天气: {info['weather']}")
        lines.append(f"- 温度: {info['temperature']}°C (体感 {info['feels_like']}°C)

```

```

        lines.append(f"- 湿度: {info['humidity']}%")
        lines.append(f"- 风速: {info['wind_speed']} m/s")
        if "rain_1h" in info:
            lines.append(f"- 最近1小时降雨量: {info['rain_1h']} mm")
        lines.append("")

    report = "\n".join(lines).strip()

    return {
        "report": report
    }

def write_file_node(state: WeatherState) -> Dict[str, Any]:
    """
    执行副作用：写文件
    """
    file_path = save_text_file(state["report"], filename="weather.txt")
    return {
        "file_path": file_path
    }

def finalize_node(state: WeatherState) -> Dict[str, Any]:
    """
    给用户最终答复
    """
    report = state.get("report", "")
    file_path = state.get("file_path")

    if file_path:
        final_answer = f"{report}\n\n✅ 已写入文件: {file_path}"
    else:
        final_answer = report

    return {
        "final_answer": final_answer
    }

# =====
# 4. 条件路由函数
# =====
def route_after_build_report(state: WeatherState) -> str:
    """
    决定 build_report 后去哪里
    """
    if state.get("need_write"):
        return "write_file"
    return "finalize"

# =====
# 5. 组装 StateGraph
# =====
def build_weather_graph():

```

```

graph_builder = StateGraph(WeatherState)

# 注册节点
graph_builder.add_node("parse_request", parse_request_node)
graph_builder.add_node("fetch_weather", fetch_weather_node)
graph_builder.add_node("build_report", build_report_node)
graph_builder.add_node("write_file", write_file_node)
graph_builder.add_node("finalize", finalize_node)

# 固定主流程
graph_builder.add_edge(START, "parse_request")
graph_builder.add_edge("parse_request", "fetch_weather")
graph_builder.add_edge("fetch_weather", "build_report")

# 条件分支: 是否写文件
graph_builder.add_conditional_edges(
    "build_report",
    route_after_build_report,
    {
        "write_file": "write_file",
        "finalize": "finalize",
    }
)

graph_builder.add_edge("write_file", "finalize")
graph_builder.add_edge("finalize", END)

return graph_builder.compile()

# =====
# 6. 运行示例
# =====
if __name__ == "__main__":
    graph = build_weather_graph()

    init_state: WeatherState = {
        "user_input": "今天上海和北京的天气怎么样，并写入本地"
    }

    result = graph.invoke(init_state)

    print("==== 最终 State =====")
    print(json.dumps(result, ensure_ascii=False, indent=2, default=str))

    print("\n==== 最终回答 =====")
    print(result["final_answer"])

```